Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Perform gray level operations images.**	
Experiment No: 7	**Date:15/08/2026**	**Enrollment No:92400133132**

Theory:
Gray level operations involve manipulating the pixel values of an image to enhance or modify its appearance. These operations are commonly used in image processing tasks such as contrast adjustment, brightness correction, and image thresholding.

Flowchart:
The flowchart for the gray level operations program will consist of the following steps:
Load the input image.

Convert the image to grayscale.
Perform the desired gray level operation.
Display the processed image.
Save the processed image (optional).

Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image directly in grayscale
src_image = cv2.imread(&#39;/content/1785219786630.jpg&#39;, cv2.IMREAD_GRAYSCALE)

# Check if image is loaded
if src_image is None:
print(&quot;Image not found!&quot;)
exit()

# -------------------------
# 1. Image Negation
# -------------------------
negative_image = 255 - src_image

# -------------------------
# 2. Thresholding
# -------------------------
_, thresholded_image = cv2.threshold(src_image, 128, 255, cv2.THRESH_BINARY)

# -------------------------
# 3. Gamma Correction
# -------------------------
gamma = 2.0
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Perform gray level operations images.**	
Experiment No: 7	**Date:15/08/2026**	**Enrollment No:92400133132**

```
# Normalize image to range [0,1]
normalized_image = src_image / 255.0

# Apply gamma correction
gamma_corrected_image = np.power(normalized_image, 1/gamma)

# Convert back to range [0,255]
gamma_corrected_image = np.uint8(gamma_corrected_image * 255)

# -------------------------
# Display Images
# -------------------------
plt.figure(figsize=(15,5))

plt.subplot(1,4,1)
plt.imshow(src_image, cmap=&#39;gray&#39;)
plt.title(&quot;Original&quot;)
plt.axis(&#39;off&#39;)

plt.subplot(1,4,2)
plt.imshow(negative_image, cmap=&#39;gray&#39;)
plt.title(&quot;Negative&quot;)
plt.axis(&#39;off&#39;)

plt.subplot(1,4,3)
plt.imshow(thresholded_image, cmap=&#39;gray&#39;)
plt.title(&quot;Threshold&quot;)
plt.axis(&#39;off&#39;)

plt.subplot(1,4,4)
plt.imshow(gamma_corrected_image, cmap=&#39;gray&#39;)
plt.title(&quot;Gamma&quot;)
plt.axis(&#39;off&#39;)

plt.show()
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Perform gray level operations images.**	
Experiment No: 7	**Date:15/08/2026**	**Enrollment No:92400133132**

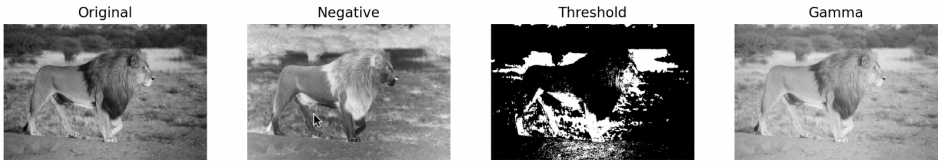
Explanation:

This experiment demonstrates three fundamental spatial-domain image processing techniques using OpenCV and Matplotlib: **Image Negation**, **Global Thresholding**, and **Gamma Correction**. Image negation inverts intensity values by subtracting each pixel from 255, transforming bright areas into dark ones and vice versa, which is often used to highlight hidden details in dark regions of an image. Global thresholding converts the grayscale image into a binary (black-and-white) image by comparing each pixel to a fixed intensity threshold of 128; pixels exceeding 128 are set to pure white (255), while all others become pure black (0), making it useful for object segmentation. Gamma correction non-linearly adjusts image contrast using power-law transformation. By setting gamma to 2.0 and raising normalized pixel values to 1/\gamma (0.5), the algorithm stretches lower-intensity values, brightening mid-tones and dark areas without saturating highlights.

The code executes these steps by first using cv2.imread() with the cv2.IMREAD_GRAYSCALE flag to load the image directly as a single-channel 2D matrix of values ranging from 0 to 255. Image negation is achieved through simple element-wise subtraction 255 - src_image. For thresholding, cv2.threshold() applies a binary mask at intensity level 128, ignoring the first returned scalar value with an underscore _. To safely compute gamma correction, the array is first divided by 255.0 to scale pixel values into a floating-point range between 0.0 and 1.0, preventing overflow during exponentiation. After applying np.power() with 1/\gamma, the output is rescaled back to the standard 8-bit range by multiplying by 255 and converting the data type back to integer format using np.uint8(). Finally, Matplotlib's plt.figure() and plt.subplot(1,4,i) functions create a clean four-panel grid, rendering each processed image alongside the original using cmap='gray' and hiding coordinate axes with plt.axis('off').

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Perform gray level operations images.**	
Experiment No: 7	**Date:15/08/2026**	**Enrollment No:92400133132**

Expected Output:





GITHUB LINK:-

https://github.com/BahopiyaDhaval/DSIP-Mu.git